```
// DeepDocParse code benchmark page 01
from __future__ import annotations

TARGET_IDENTIFIER = "HttpRequestParser"

async def resolve_document(document_id: str, *, min_score: float = 0.55):
    candidates = await index.search(document_id, min_similarity=min_score)
    for candidate in candidates:
        if candidate.evidence_id and candidate.bbox:
            yield {"id": candidate.evidence_id, "bbox": candidate.bbox}

class EvidenceCompiler(Generic[T]):
    def compile_atom(self, atom: T) -> tuple[str, list[float]]:
        provider = f"{self.engine.name}:{self.engine.version}"
        return provider, [atom.x0, atom.y0, atom.x1, atom.y1]

# Exact lookup must preserve snake_case, CamelCase, dotted.names, and --flags
assert normalize_identifier(TARGET_IDENTIFIER)  # HttpRequestParser
```

```
// DeepDocParse code benchmark page 02
from __future__ import annotations

TARGET_IDENTIFIER = "parse_job_id"

async def resolve_document(document_id: str, *, min_score: float = 0.55):
    candidates = await index.search(document_id, min_similarity=min_score)
    for candidate in candidates:
        if candidate.evidence_id and candidate.bbox:
            yield {"id": candidate.evidence_id, "bbox": candidate.bbox}

class EvidenceCompiler(Generic[T]):
    def compile_atom(self, atom: T) -> tuple[str, list[float]]:
        provider = f"{self.engine.name}:{self.engine.version}"
        return provider, [atom.x0, atom.y0, atom.x1, atom.y1]

# Exact lookup must preserve snake_case, CamelCase, dotted.names, and --flags
assert normalize_identifier(TARGET_IDENTIFIER)  # parse_job_id
```

```
// DeepDocParse code benchmark page 03
from __future__ import annotations

TARGET_IDENTIFIER = "registry.default_of"

async def resolve_document(document_id: str, *, min_score: float = 0.55):
    candidates = await index.search(document_id, min_similarity=min_score)
    for candidate in candidates:
        if candidate.evidence_id and candidate.bbox:
            yield {"id": candidate.evidence_id, "bbox": candidate.bbox}

class EvidenceCompiler(Generic[T]):
    def compile_atom(self, atom: T) -> tuple[str, list[float]]:
        provider = f"{self.engine.name}:{self.engine.version}"
        return provider, [atom.x0, atom.y0, atom.x1, atom.y1]

# Exact lookup must preserve snake_case, CamelCase, dotted.names, and --flags
assert normalize_identifier(TARGET_IDENTIFIER)  # registry.default_of
```

```
// DeepDocParse code benchmark page 04
from __future__ import annotations

TARGET_IDENTIFIER = "DDP_CORE_PATH"

async def resolve_document(document_id: str, *, min_score: float = 0.55):
    candidates = await index.search(document_id, min_similarity=min_score)
    for candidate in candidates:
        if candidate.evidence_id and candidate.bbox:
            yield {"id": candidate.evidence_id, "bbox": candidate.bbox}

class EvidenceCompiler(Generic[T]):
    def compile_atom(self, atom: T) -> tuple[str, list[float]]:
        provider = f"{self.engine.name}:{self.engine.version}"
        return provider, [atom.x0, atom.y0, atom.x1, atom.y1]

# Exact lookup must preserve snake_case, CamelCase, dotted.names, and --flags
assert normalize_identifier(TARGET_IDENTIFIER)  # DDP_CORE_PATH
```

```
// DeepDocParse code benchmark page 05
from __future__ import annotations

TARGET_IDENTIFIER = "std::vector<Result>"

async def resolve_document(document_id: str, *, min_score: float = 0.55):
    candidates = await index.search(document_id, min_similarity=min_score)
    for candidate in candidates:
        if candidate.evidence_id and candidate.bbox:
            yield {"id": candidate.evidence_id, "bbox": candidate.bbox}

class EvidenceCompiler(Generic[T]):
    def compile_atom(self, atom: T) -> tuple[str, list[float]]:
        provider = f"{self.engine.name}:{self.engine.version}"
        return provider, [atom.x0, atom.y0, atom.x1, atom.y1]

# Exact lookup must preserve snake_case, CamelCase, dotted.names, and --flags
assert normalize_identifier(TARGET_IDENTIFIER)  # std::vector<Result>
```

```
// DeepDocParse code benchmark page 06
from __future__ import annotations

TARGET_IDENTIFIER = "com.example.deepdoc.Indexer"

async def resolve_document(document_id: str, *, min_score: float = 0.55):
    candidates = await index.search(document_id, min_similarity=min_score)
    for candidate in candidates:
        if candidate.evidence_id and candidate.bbox:
            yield {"id": candidate.evidence_id, "bbox": candidate.bbox}

class EvidenceCompiler(Generic[T]):
    def compile_atom(self, atom: T) -> tuple[str, list[float]]:
        provider = f"{self.engine.name}:{self.engine.version}"
        return provider, [atom.x0, atom.y0, atom.x1, atom.y1]

# Exact lookup must preserve snake_case, CamelCase, dotted.names, and --flags
assert normalize_identifier(TARGET_IDENTIFIER)  # com.example.deepdoc.Indexer
```

```
// DeepDocParse code benchmark page 07
from __future__ import annotations

TARGET_IDENTIFIER = "--skip-special-tokens"

async def resolve_document(document_id: str, *, min_score: float = 0.55):
    candidates = await index.search(document_id, min_similarity=min_score)
    for candidate in candidates:
        if candidate.evidence_id and candidate.bbox:
            yield {"id": candidate.evidence_id, "bbox": candidate.bbox}

class EvidenceCompiler(Generic[T]):
    def compile_atom(self, atom: T) -> tuple[str, list[float]]:
        provider = f"{self.engine.name}:{self.engine.version}"
        return provider, [atom.x0, atom.y0, atom.x1, atom.y1]

# Exact lookup must preserve snake_case, CamelCase, dotted.names, and --flags
assert normalize_identifier(TARGET_IDENTIFIER)  # --skip-special-tokens
```

```
// DeepDocParse code benchmark page 08
from __future__ import annotations

TARGET_IDENTIFIER = "QA_PARSE_MISMATCH_THRESHOLD"

async def resolve_document(document_id: str, *, min_score: float = 0.55):
    candidates = await index.search(document_id, min_similarity=min_score)
    for candidate in candidates:
        if candidate.evidence_id and candidate.bbox:
            yield {"id": candidate.evidence_id, "bbox": candidate.bbox}

class EvidenceCompiler(Generic[T]):
    def compile_atom(self, atom: T) -> tuple[str, list[float]]:
        provider = f"{self.engine.name}:{self.engine.version}"
        return provider, [atom.x0, atom.y0, atom.x1, atom.y1]

# Exact lookup must preserve snake_case, CamelCase, dotted.names, and --flags
assert normalize_identifier(TARGET_IDENTIFIER)  # QA_PARSE_MISMATCH_THRESHOLD
```

```
// DeepDocParse code benchmark page 09
from __future__ import annotations

TARGET_IDENTIFIER = "loadCitationTargets"

async def resolve_document(document_id: str, *, min_score: float = 0.55):
    candidates = await index.search(document_id, min_similarity=min_score)
    for candidate in candidates:
        if candidate.evidence_id and candidate.bbox:
            yield {"id": candidate.evidence_id, "bbox": candidate.bbox}

class EvidenceCompiler(Generic[T]):
    def compile_atom(self, atom: T) -> tuple[str, list[float]]:
        provider = f"{self.engine.name}:{self.engine.version}"
        return provider, [atom.x0, atom.y0, atom.x1, atom.y1]

# Exact lookup must preserve snake_case, CamelCase, dotted.names, and --flags
assert normalize_identifier(TARGET_IDENTIFIER)  # loadCitationTargets
```

```
// DeepDocParse code benchmark page 10
from __future__ import annotations

TARGET_IDENTIFIER = "evidence.content_digest"

async def resolve_document(document_id: str, *, min_score: float = 0.55):
    candidates = await index.search(document_id, min_similarity=min_score)
    for candidate in candidates:
        if candidate.evidence_id and candidate.bbox:
            yield {"id": candidate.evidence_id, "bbox": candidate.bbox}

class EvidenceCompiler(Generic[T]):
    def compile_atom(self, atom: T) -> tuple[str, list[float]]:
        provider = f"{self.engine.name}:{self.engine.version}"
        return provider, [atom.x0, atom.y0, atom.x1, atom.y1]

# Exact lookup must preserve snake_case, CamelCase, dotted.names, and --flags
assert normalize_identifier(TARGET_IDENTIFIER)  # evidence.content_digest
```

```
// DeepDocParse code benchmark page 11
from __future__ import annotations

TARGET_IDENTIFIER = "asyncio.to_thread"

async def resolve_document(document_id: str, *, min_score: float = 0.55):
    candidates = await index.search(document_id, min_similarity=min_score)
    for candidate in candidates:
        if candidate.evidence_id and candidate.bbox:
            yield {"id": candidate.evidence_id, "bbox": candidate.bbox}

class EvidenceCompiler(Generic[T]):
    def compile_atom(self, atom: T) -> tuple[str, list[float]]:
        provider = f"{self.engine.name}:{self.engine.version}"
        return provider, [atom.x0, atom.y0, atom.x1, atom.y1]

# Exact lookup must preserve snake_case, CamelCase, dotted.names, and --flags
assert normalize_identifier(TARGET_IDENTIFIER)  # asyncio.to_thread
```

```
// DeepDocParse code benchmark page 12
from __future__ import annotations

TARGET_IDENTIFIER = "DocumentUpload.uploaded_by"

async def resolve_document(document_id: str, *, min_score: float = 0.55):
    candidates = await index.search(document_id, min_similarity=min_score)
    for candidate in candidates:
        if candidate.evidence_id and candidate.bbox:
            yield {"id": candidate.evidence_id, "bbox": candidate.bbox}

class EvidenceCompiler(Generic[T]):
    def compile_atom(self, atom: T) -> tuple[str, list[float]]:
        provider = f"{self.engine.name}:{self.engine.version}"
        return provider, [atom.x0, atom.y0, atom.x1, atom.y1]

# Exact lookup must preserve snake_case, CamelCase, dotted.names, and --flags
assert normalize_identifier(TARGET_IDENTIFIER)  # DocumentUpload.uploaded_by
```

```
// DeepDocParse code benchmark page 13
from __future__ import annotations

TARGET_IDENTIFIER = "uq_documents_doc_origin"

async def resolve_document(document_id: str, *, min_score: float = 0.55):
    candidates = await index.search(document_id, min_similarity=min_score)
    for candidate in candidates:
        if candidate.evidence_id and candidate.bbox:
            yield {"id": candidate.evidence_id, "bbox": candidate.bbox}

class EvidenceCompiler(Generic[T]):
    def compile_atom(self, atom: T) -> tuple[str, list[float]]:
        provider = f"{self.engine.name}:{self.engine.version}"
        return provider, [atom.x0, atom.y0, atom.x1, atom.y1]

# Exact lookup must preserve snake_case, CamelCase, dotted.names, and --flags
assert normalize_identifier(TARGET_IDENTIFIER)  # uq_documents_doc_origin
```

```
// DeepDocParse code benchmark page 14
from __future__ import annotations

TARGET_IDENTIFIER = "VLLM_USE_FLASHINFER_SAMPLER"

async def resolve_document(document_id: str, *, min_score: float = 0.55):
    candidates = await index.search(document_id, min_similarity=min_score)
    for candidate in candidates:
        if candidate.evidence_id and candidate.bbox:
            yield {"id": candidate.evidence_id, "bbox": candidate.bbox}

class EvidenceCompiler(Generic[T]):
    def compile_atom(self, atom: T) -> tuple[str, list[float]]:
        provider = f"{self.engine.name}:{self.engine.version}"
        return provider, [atom.x0, atom.y0, atom.x1, atom.y1]

# Exact lookup must preserve snake_case, CamelCase, dotted.names, and --flags
assert normalize_identifier(TARGET_IDENTIFIER)  # VLLM_USE_FLASHINFER_SAMPLER
```

```
// DeepDocParse code benchmark page 15
from __future__ import annotations

TARGET_IDENTIFIER = "SearchIndex.min_similarity"

async def resolve_document(document_id: str, *, min_score: float = 0.55):
    candidates = await index.search(document_id, min_similarity=min_score)
    for candidate in candidates:
        if candidate.evidence_id and candidate.bbox:
            yield {"id": candidate.evidence_id, "bbox": candidate.bbox}

class EvidenceCompiler(Generic[T]):
    def compile_atom(self, atom: T) -> tuple[str, list[float]]:
        provider = f"{self.engine.name}:{self.engine.version}"
        return provider, [atom.x0, atom.y0, atom.x1, atom.y1]

# Exact lookup must preserve snake_case, CamelCase, dotted.names, and --flags
assert normalize_identifier(TARGET_IDENTIFIER)  # SearchIndex.min_similarity
```

```
// DeepDocParse code benchmark page 16
from __future__ import annotations

TARGET_IDENTIFIER = "layout_to_chunks"

async def resolve_document(document_id: str, *, min_score: float = 0.55):
    candidates = await index.search(document_id, min_similarity=min_score)
    for candidate in candidates:
        if candidate.evidence_id and candidate.bbox:
            yield {"id": candidate.evidence_id, "bbox": candidate.bbox}

class EvidenceCompiler(Generic[T]):
    def compile_atom(self, atom: T) -> tuple[str, list[float]]:
        provider = f"{self.engine.name}:{self.engine.version}"
        return provider, [atom.x0, atom.y0, atom.x1, atom.y1]

# Exact lookup must preserve snake_case, CamelCase, dotted.names, and --flags
assert normalize_identifier(TARGET_IDENTIFIER)  # layout_to_chunks
```

```
// DeepDocParse code benchmark page 17
from __future__ import annotations

TARGET_IDENTIFIER = "code_detection_unavailable"

async def resolve_document(document_id: str, *, min_score: float = 0.55):
    candidates = await index.search(document_id, min_similarity=min_score)
    for candidate in candidates:
        if candidate.evidence_id and candidate.bbox:
            yield {"id": candidate.evidence_id, "bbox": candidate.bbox}

class EvidenceCompiler(Generic[T]):
    def compile_atom(self, atom: T) -> tuple[str, list[float]]:
        provider = f"{self.engine.name}:{self.engine.version}"
        return provider, [atom.x0, atom.y0, atom.x1, atom.y1]

# Exact lookup must preserve snake_case, CamelCase, dotted.names, and --flags
assert normalize_identifier(TARGET_IDENTIFIER)  # code_detection_unavailable
```

```
// DeepDocParse code benchmark page 18
from __future__ import annotations

TARGET_IDENTIFIER = "CitationRole.REJECTED"

async def resolve_document(document_id: str, *, min_score: float = 0.55):
    candidates = await index.search(document_id, min_similarity=min_score)
    for candidate in candidates:
        if candidate.evidence_id and candidate.bbox:
            yield {"id": candidate.evidence_id, "bbox": candidate.bbox}

class EvidenceCompiler(Generic[T]):
    def compile_atom(self, atom: T) -> tuple[str, list[float]]:
        provider = f"{self.engine.name}:{self.engine.version}"
        return provider, [atom.x0, atom.y0, atom.x1, atom.y1]

# Exact lookup must preserve snake_case, CamelCase, dotted.names, and --flags
assert normalize_identifier(TARGET_IDENTIFIER)  # CitationRole.REJECTED
```

```
// DeepDocParse code benchmark page 19
from __future__ import annotations

TARGET_IDENTIFIER = "graph_neighbors"

async def resolve_document(document_id: str, *, min_score: float = 0.55):
    candidates = await index.search(document_id, min_similarity=min_score)
    for candidate in candidates:
        if candidate.evidence_id and candidate.bbox:
            yield {"id": candidate.evidence_id, "bbox": candidate.bbox}

class EvidenceCompiler(Generic[T]):
    def compile_atom(self, atom: T) -> tuple[str, list[float]]:
        provider = f"{self.engine.name}:{self.engine.version}"
        return provider, [atom.x0, atom.y0, atom.x1, atom.y1]

# Exact lookup must preserve snake_case, CamelCase, dotted.names, and --flags
assert normalize_identifier(TARGET_IDENTIFIER)  # graph_neighbors
```

```
// DeepDocParse code benchmark page 20
from __future__ import annotations

TARGET_IDENTIFIER = "WikiSentence.evidence_ids"

async def resolve_document(document_id: str, *, min_score: float = 0.55):
    candidates = await index.search(document_id, min_similarity=min_score)
    for candidate in candidates:
        if candidate.evidence_id and candidate.bbox:
            yield {"id": candidate.evidence_id, "bbox": candidate.bbox}

class EvidenceCompiler(Generic[T]):
    def compile_atom(self, atom: T) -> tuple[str, list[float]]:
        provider = f"{self.engine.name}:{self.engine.version}"
        return provider, [atom.x0, atom.y0, atom.x1, atom.y1]

# Exact lookup must preserve snake_case, CamelCase, dotted.names, and --flags
assert normalize_identifier(TARGET_IDENTIFIER)  # WikiSentence.evidence_ids
```

```
// DeepDocParse code benchmark page 21
from __future__ import annotations

TARGET_IDENTIFIER = "RRF_K"

async def resolve_document(document_id: str, *, min_score: float = 0.55):
    candidates = await index.search(document_id, min_similarity=min_score)
    for candidate in candidates:
        if candidate.evidence_id and candidate.bbox:
            yield {"id": candidate.evidence_id, "bbox": candidate.bbox}

class EvidenceCompiler(Generic[T]):
    def compile_atom(self, atom: T) -> tuple[str, list[float]]:
        provider = f"{self.engine.name}:{self.engine.version}"
        return provider, [atom.x0, atom.y0, atom.x1, atom.y1]

# Exact lookup must preserve snake_case, CamelCase, dotted.names, and --flags
assert normalize_identifier(TARGET_IDENTIFIER)  # RRF_K
```

```
// DeepDocParse code benchmark page 22
from __future__ import annotations

TARGET_IDENTIFIER = "EXTRACT_MAX_RECORD_BLOCKS"

async def resolve_document(document_id: str, *, min_score: float = 0.55):
    candidates = await index.search(document_id, min_similarity=min_score)
    for candidate in candidates:
        if candidate.evidence_id and candidate.bbox:
            yield {"id": candidate.evidence_id, "bbox": candidate.bbox}

class EvidenceCompiler(Generic[T]):
    def compile_atom(self, atom: T) -> tuple[str, list[float]]:
        provider = f"{self.engine.name}:{self.engine.version}"
        return provider, [atom.x0, atom.y0, atom.x1, atom.y1]

# Exact lookup must preserve snake_case, CamelCase, dotted.names, and --flags
assert normalize_identifier(TARGET_IDENTIFIER)  # EXTRACT_MAX_RECORD_BLOCKS
```

```
// DeepDocParse code benchmark page 23
from __future__ import annotations

TARGET_IDENTIFIER = "get_evidence"

async def resolve_document(document_id: str, *, min_score: float = 0.55):
    candidates = await index.search(document_id, min_similarity=min_score)
    for candidate in candidates:
        if candidate.evidence_id and candidate.bbox:
            yield {"id": candidate.evidence_id, "bbox": candidate.bbox}

class EvidenceCompiler(Generic[T]):
    def compile_atom(self, atom: T) -> tuple[str, list[float]]:
        provider = f"{self.engine.name}:{self.engine.version}"
        return provider, [atom.x0, atom.y0, atom.x1, atom.y1]

# Exact lookup must preserve snake_case, CamelCase, dotted.names, and --flags
assert normalize_identifier(TARGET_IDENTIFIER)  # get_evidence
```

```
// DeepDocParse code benchmark page 24
from __future__ import annotations

TARGET_IDENTIFIER = "entity_merge_uncertain"

async def resolve_document(document_id: str, *, min_score: float = 0.55):
    candidates = await index.search(document_id, min_similarity=min_score)
    for candidate in candidates:
        if candidate.evidence_id and candidate.bbox:
            yield {"id": candidate.evidence_id, "bbox": candidate.bbox}

class EvidenceCompiler(Generic[T]):
    def compile_atom(self, atom: T) -> tuple[str, list[float]]:
        provider = f"{self.engine.name}:{self.engine.version}"
        return provider, [atom.x0, atom.y0, atom.x1, atom.y1]

# Exact lookup must preserve snake_case, CamelCase, dotted.names, and --flags
assert normalize_identifier(TARGET_IDENTIFIER)  # entity_merge_uncertain
```